Exercício 11.3: Depuração: Application `OutOfSync` e `Degraded`

Aluno: Luiz Brito · **Data:** 25/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

Mesmo ambiente do 11.1 e do 11.2: kind com Kubernetes 1.36.1, Argo CD v3.5.6, repositório `fiap-store-config` no Gitea do cluster e Applications `fiap-store-dev` / `fiap-store-prod` no projeto `fiap-store`. Toda correção foi feita **pelo Git**. No cluster só foram executados o drift proposital do cenário A, a reaplicação declarativa da Application e a instalação do controller do Sealed Secrets. Sequência em `05-commands.sh`.

Cenário A: drift manual

Sintoma (com `selfHeal: true`):

```
21:25:16.131 inicio
$ kubectl scale deploy/store-api -n store-dev --replicas=1
deployment.apps/store-api scaled
$ kubectl set image deploy/store-api store-api=nginx:1.27 -n store-dev
deployment.apps/store-api image updated
21:25:16.346 drift aplicado            → replicas=1, image=nginx:1.27

$ argocd app diff fiap-store-dev        (21:25:16.644, antes da reconciliação)
===== apps/Deployment store-dev/store-api ======
183c183
<   replicas: 1
---
>   replicas: 4
203c203
<         image: nginx:1.27
---
>         image: store-api:1.1

21:25:19  4 store-api:1.1
21:25:22  4 store-api:1.1
```

Diagnóstico (log do `argocd-application-controller`):

```
2026-10-25T21:25:16Z Updated sync status: Synced → OutOfSync
2026-10-25T21:25:16Z Initiated automated sync to 'a119cc203634c77c7617e93cb500275d95fc4df2'
2026-10-25T21:25:16Z Updated sync status: OutOfSync → Synced
```

O Argo desfez **as duas** alterações em menos de 3 s. O controller observa os recursos por watch, então a mudança no cluster dispara a comparação na hora, sem esperar o polling do Git. O diff mostra o lado vivo (`<`) contra o Git (`>`).

Com o selfHeal desligado:

```
$ argocd app set fiap-store-dev --sync-policy none --grpc-web
syncPolicy={"syncOptions":["CreateNamespace=true"]}
21:25:44 drift (mesmos dois comandos)
21:28:14 após 150 s
$ kubectl -n store-dev get deploy store-api -o wide
NAME        READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES
store-api   1/1     1            1           40h   store-api    nginx:1.27
$ argocd app get fiap-store-dev
Sync Policy:        Manual
Sync Status:        OutOfSync from main (a119cc2)
Health Status:      Progressing
$ kubectl -n store-dev get pods
store-api-5846c7bc85-mswv9   0/1     CrashLoopBackOff   4 (60s ago)   2m30s
store-api-59c6bc8b55-sss2f   1/1     Running            0             40h
```

Diferença: sem `selfHeal`, o Argo **detecta** o drift (`OutOfSync`) mas **não age**, e o cluster fica divergente do Git até alguém sincronizar. O `scale` derrubou 3 das 4 réplicas na hora. O `set image` não derrubou o serviço por acaso: o nginx entra em `CrashLoopBackOff` com o `securityContext` da `store-api` (UID 65532, raiz somente-leitura), e o `maxUnavailable: 0` manteve o Pod antigo vivo. Com `selfHeal`, o Git é a fonte da verdade de forma **ativa**. Sem ele, é só **observada**.

Causa raiz: alteração manual no cluster, fora do Git.

Correção: reaplicar a Application declarativa (`syncPolicy.automated` com `selfHeal: true`), que está versionada em `../11.2/02-application-dev.yaml`. A mudança é na configuração do Argo, não nos recursos:

```
$ kubectl apply -f ../11.2/02-application-dev.yaml
application.argoproj.io/fiap-store-dev configured
$ kubectl -n store-dev get deploy store-api -o wide     (21:28:53)
store-api   4/4     4            4           40h   store-api    store-api:1.1
Sync Policy:        Automated (Prune)
Sync Status:        Synced to main (a119cc2)
Health Status:      Healthy
```

Cenário B: `Synced` + `Degraded`

Preparação pelo Git: commit `5fc62d4` com `progressDeadlineSeconds: 90` na base. O padrão é 600 s, e um rollout travado só vira `Degraded` depois desse prazo. O commit seguinte, `c59dcde`, troca a tag de dev para `9.9-inexistente`.

Sintoma:

```
21:29:20 push   →   21:30:54 Degraded
$ argocd app get fiap-store-dev
Sync Status:        Synced to main (c59dcde)
Health Status:      Degraded
GROUP  KIND        NAMESPACE  NAME       STATUS  HEALTH    HOOK  MESSAGE
apps   Deployment  store-dev  store-api  Synced  Degraded        deployment.apps/store-api configured
       Service     store-dev  store-api  Synced  Healthy

$ kubectl -n store-dev get pods
store-api-59c6bc8b55-gk6xv   1/1     Running            0          2m12s
store-api-59c6bc8b55-mp8q5   1/1     Running            0          2m26s
store-api-59c6bc8b55-sss2f   1/1     Running            0          40h
store-api-59c6bc8b55-z7dc2   1/1     Running            0          2m23s
store-api-64fffc7f78-442r6   0/1     ImagePullBackOff   0          93s
```

Diagnóstico:

```
$ kubectl -n store-dev describe pod store-api-64fffc7f78-442r6
    Image:          store-api:9.9-inexistente
    State:          Waiting
      Reason:       ImagePullBackOff
  Warning  Failed  50s (x3 over 92s)  kubelet  Failed to pull image "store-api:9.9-inexistente":
failed to pull and unpack image
           "docker.io/library/store-api:9.9-inexistente": failed to resolve reference ...: pull
access denied, repository does not exist
           or may require authorization: server message: insufficient_scope: authorization failed

$ kubectl -n store-dev get deploy store-api -o jsonpath='{range .status.conditions[*]}...'
Available=True MinimumReplicasAvailable: Deployment has minimum availability.
Progressing=False ProgressDeadlineExceeded: ReplicaSet "store-api-64fffc7f78" has timed out
progressing.

controller: 21:29:21Z Updated health status: Healthy → Progressing
            21:30:52Z Updated health status: Progressing → Degraded
```

Por que não é bug do Argo: *sync* e *health* respondem a perguntas diferentes. O **sync** compara o Git com o cluster: o Deployment no cluster tem exatamente a tag `9.9-inexistente` que está no Git, então ele está `Synced`. O Argo aplicou fielmente o que foi pedido. O **health** avalia se os recursos funcionam: para um Deployment, o Argo lê a condição `Progressing=False / ProgressDeadlineExceeded`, e daí o `Degraded`. Um commit errado aplicado corretamente é exatamente essa combinação. O problema está no conteúdo do Git, não na entrega.

Causa raiz: tag de imagem inexistente commitada. Os 4 Pods antigos seguiram atendendo graças ao `maxUnavailable: 0`.

Correção:

```
$ git revert --no-edit HEAD
[main 9455bbf] Revert "chore(dev): bump store-api to 9.9-inexistente"
-    newTag: "9.9-inexistente"
+    newTag: "1.1"
$ git push
$ argocd app get fiap-store-dev
Sync Status:        Synced to main (9455bbf)
Health Status:      Healthy
controller: 21:31:09Z Updated health status: Degraded → Healthy
```

O revert é um commit novo e não reescreve o histórico. O commit errado continua auditável.

Cenário C: dependência ausente, resolvida com Sealed Secrets

Commit `6573219` **:** novo Deployment na base, consumindo um Secret que não existe e que corretamente não está no Git.

```
# consome a senha do banco via secretKeyRef; o Secret NAO esta no Git
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-relatorio
spec:
  replicas: 1
  selector:
    matchLabels:
      app: store-relatorio
  template:
    metadata:
      labels:
        app: store-relatorio
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: relatorio
          image: busybox:1.37
          command: ["sh", "-c", "echo conectado ao banco com senha de ${#DB_PASSWORD} caracteres;
sleep 3600"]
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: store-db-credentials
                  key: DB_PASSWORD
          resources:
            requests:
              cpu: 10m
              memory: 16Mi
            limits:
              memory: 32Mi
          securityContext:
            allowPrivilegeEscalation: false
```

```
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

Sintoma e diagnóstico:

```
$ kubectl -n store-dev get pods -l app=store-relatorio
store-relatorio-685cf86b68-hlts7   0/1     CreateContainerConfigError   0          25s

$ kubectl -n store-dev describe pod -l app=store-relatorio
    State:          Waiting
      Reason:       CreateContainerConfigError
  Warning  Failed  7s (x5 over 34s)  kubelet  spec.containers{relatorio}: Error: secret "store-db-
credentials" not found

$ kubectl -n store-dev get secret store-db-credentials
Error from server (NotFound): secrets "store-db-credentials" not found

$ argocd app get fiap-store-dev
Sync Status:        Synced to main (6573219)
Health Status:      Progressing
apps   Deployment  store-dev  store-relatorio  Synced  Progressing        deployment.apps/store-
relatorio created
```

De novo `Synced`: o Deployment foi aplicado, e o que falta é um objeto que nunca esteve no Git.

Causa raiz: o segredo é uma dependência de execução sem um jeito seguro de ser versionado.

Correção: controller do Sealed Secrets e `SealedSecret` por namespace, commitado em `overlays/dev` e `overlays/prod` (`d34967d`).

O release mais novo (v0.41.1) ainda não tem imagem publicada: `docker.io/bitnami/sealed-secrets-controller:0.41.1: not found` (nem no ghcr). Por isso foi usado o **v0.40.0**, a última versão com imagem, no controller e no `kubeseal`, com checksum conferido.

```
$ kubectl create secret generic store-db-credentials -n store-dev --from-literal=DB_PASSWORD="$SENHA"
\
    --dry-run=client -o yaml | kubeseal --format yaml --controller-namespace kube-system \
    > overlays/dev/sealed-db-credentials.yaml
```

```
---
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: store-db-credentials
  namespace: store-dev
spec:
  encryptedData:
    DB_PASSWORD:
AgCSumaksJuiRE2kScFF1fGvlChMsAluFLSFq82O2F7lc9EVgEaGIgYnGmiSyZeCmczixhqCJINZHRc5PNttwX54NA0Ed9iNU+t0q
Z79x31efIXzo8h94SAYd/oO0VNaz0iWYmiZh9XTW+FLC1pIXHMEcJPrvvtfW/Ogj7ZiaPok2FcVkRbN1a1lOXMe5P0gIO+4gdrZYm
lAQZE5Tmy7Ds+UBEDwgS2CbJnTkdNYMzuatPfoHpE9WIJXsRqU11HV0jyyGPGSYqOW3KESLqFORGI3RFF4YW7Rh8STl84EqndjvQz
/MuoWBqnaYgO3AtlDGSfkH+KhRAN0xghzs5hVV2K8yMzcGckSSPgQYZh650FYHXyUpDwMOUSiVyIq+26YwVRgn7RfuyAJbeNSbWpf
jyFVedx6510fMSqIAWDQStnF527kjtWl6ycqh6Sp4emMfCfnuJlOTz7RRv8hGano9Z+Cqhn73B3jDlDTVl8R5P+gNs4QR6UTheP3T
fYMuC+JWRldn6Ot/yRO8oS0+Ind+7Fjeka0U/U93Nf8hc0IHFedr0NETnTmhSs1SLMjOn2Py7nijkFcFOtM5y9GgAScC9RTYR4fsV
f3HHGywW7cbvPNbHHTcIah9wT3BS0v8KjQW1ze3XAd0RCG0HtcWbtgHzDRpB5NUAyKZo+iVGSuHx4jPFLZTQoZBLGJzbAhGQPC8ad
O4i+1qqPnPIJp/FBUxI1e493nEw==
  template:
    metadata:
      name: store-db-credentials
      namespace: store-dev
```

O `SealedSecret` é cifrado com a chave pública do controller e vale **só** para `store-dev/store-db-credentials` (escopo `strict`), por isso há um arquivo por namespace. Copiá-lo para outro namespace não serve para nada.

A senha não está no repositório:

```
$ grep -ri "$SENHA" . --exclude-dir=.git
grep exit=1
$ grep -ri "$(echo -n $SENHA | base64)" . --exclude-dir=.git
grep exit=1
```

Também foi testado o base64, que é o que um `Secret` comum commitado exporia.

Evidência da correção:

```
$ kubectl get secret store-db-credentials -n store-dev
NAME                    TYPE     DATA   AGE
store-db-credentials    Opaque   1      4s
$ kubectl get secret store-db-credentials -n store-dev -o
jsonpath='{.metadata.ownerReferences[0].kind}/...'
SealedSecret/store-db-credentials
$ kubectl -n store-dev get pods -l app=store-relatorio
store-relatorio-685cf86b68-hlts7   1/1     Running   0          57s
$ kubectl -n store-dev logs deploy/store-relatorio
conectado ao banco com senha de 20 caracteres
$ argocd app get fiap-store-dev / fiap-store-prod
Sync Status:        Synced to main (d34967d)     Health Status:      Healthy
Sync Status:        Synced to main (d34967d)     Health Status:      Healthy
```

O Secret real tem o `SealedSecret` como dono, então apagar o `SealedSecret` (por exemplo, num prune do Argo) apaga o Secret junto.

Cenário D (bônus): falha por RBAC do AppProject

```yaml
# bonus D: mesmo projeto, mas sem Namespace no clusterResourceWhitelist
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: fiap-store-sem-ns
  namespace: argocd
spec:
  sourceRepos:
    - http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
  destinations:
    - server: https://kubernetes.default.svc
      namespace: store-qa
  clusterResourceWhitelist: []
```

Application `fiap-store-qa` (`02-application-store-qa.yaml`) com destino `store-qa` e `CreateNamespace=true`:

```
$ argocd app get fiap-store-qa
Sync Status:        OutOfSync from main (d34967d)
Health Status:      Missing
GROUP  KIND        NAMESPACE  NAME             STATUS     HEALTH      HOOK  MESSAGE
       Namespace               store-qa                    SyncFailed        resource :Namespace is
not permitted in project fiap-store-sem-ns
apps   Deployment  store-qa   store-api        OutOfSync  Missing

operationState: Running: one or more synchronization tasks are not valid: resource :Namespace is not
permitted in project fiap-store-sem-ns. Retrying attempt #2

controller: "Adding resource result, status: 'SyncFailed', ... message: 'resource :Namespace is not
permitted in project fiap-store-sem-ns'", "phase":"PreSync"
```

O `CreateNamespace=true` não é mágico: ele vira um recurso `Namespace` no sync, sujeito ao `clusterResourceWhitelist` do projeto. Como o Namespace falha na fase `PreSync`, **nada** é aplicado. Por isso o `fiap-store` do 11.2 lista explicitamente `Namespace`. A Application e o projeto de teste foram apagados depois.

Critérios de aceite

Critério	Resultado
A, B e C reproduzidos e terminando em `Synced/Healthy`	ok
B explica sync (Git x cluster) x health (funcionamento)	ok
`SealedSecret` no repositório sem a senha (`grep -i` vazio)	ok (texto puro e base64)
`kubectl get secret store-db-credentials -n store-dev`	ok